

Module 1 – Practice Exercises

Level 1 – Understanding

Q1.

The two file-system problems are data redundancy/duplication and data inconsistency. The same customer address is stored in two independent copies, so updating one copy does not automatically update the other. A database stores the shared customer data in one controlled location and lets authorized applications access the same record, reducing duplicate copies and keeping updates consistent.

Q2.

Three things a DBMS provides beyond a basic file system are:

1. Concurrency control – it coordinates simultaneous transactions. Without it, two ATMs could update the same balance incorrectly.
2. Constraint enforcement – it can enforce rules such as unique account numbers and valid customer references. Without it, invalid or orphan banking records could enter the data.
3. Transaction/recovery management – it can commit or roll back a group of operations and recover after failures. Without it, a transfer could debit one account but fail to credit the other, leaving the data incorrect.

Q3.

| Formal relational term | Everyday term | ICICI Bank example |

| Relation | Table | customer table |

| Tuple | Row / record | One customer's complete record |

| Attribute | Column / field | customer.email |

| Degree | Number of columns | Number of attributes in customer |

| Cardinality | Number of rows | Number of customer records |

The key distinction is that degree counts columns, while cardinality counts rows.

Q4.

1. customer.customer_id – Primary key.
2. account.account_number – Candidate key (and, if declared as the table's PRIMARY KEY in the supplied schema, it is the primary key). Treat it according to the actual constraint definition in the database.
3. account.customer_id – Foreign key.
4. account.balance – None of these.
5. employee.manager_id – Foreign key (self-referencing, if it references employee.employee_id/manager_id as defined by the schema).

6. branch.ifsc_code – Candidate key if it is declared UNIQUE and identifies a branch; otherwise it is not necessarily a key. Check the supplied database constraint definition before submitting a categorical answer.

Q5.

1. OLTP – A balance check is a small, real-time operational transaction/query.
2. OLAP – It aggregates historical loan data by branch for analysis.
3. OLTP – An ATM withdrawal changes an account's operational balance.
4. OLAP – It scans and aggregates a large historical transaction set.
5. OLTP – Opening an account is an operational write involving a small number of records.
6. OLAP – Marketing is ranking customers using aggregated historical/monthly data.

Q6.

Running a large quarterly regulatory report directly on the core banking database can compete with customer-facing transactions for CPU, memory, I/O and locks. Complex scans and aggregations can also increase latency for OLTP workloads and make peak-period performance less predictable. Finally, analytical/reporting workloads often need different indexing, partitioning and retention patterns than the core system. I would offer a reporting replica/data warehouse or other dedicated analytical store fed from the core database, with controlled refresh and reconciliation procedures.

Q7.

Atomicity → d. Money is debited but never credited.

Consistency → c. An account exists for a customer who does not.

Isolation → b. Two ATMs both pay out against the same balance.

Durability → a. A committed transfer disappears after a power cut.

Q8.

The write-ahead log (WAL) is persisted before the corresponding changed data page so that, after a crash, PostgreSQL can replay the logged change and recover the database to a consistent state. If the data page were written first and the WAL record were not safely persisted, a crash could leave a changed page without the recovery information needed to establish what happened, weakening crash recovery and durability.

Q9.

This breaks physical data independence at the application/interface level by making the application depend on the table's complete physical column layout rather than explicitly requesting the fields it needs. When a pan_number column is later added, SELECT * changes the returned column set, which can break code that assumes a fixed number, order, or mapping of columns. Explicitly naming required columns avoids this dependency.

Q10.

Oracle – Core banking/enterprise workloads where the bank uses Oracle for mission-critical relational processing.

PostgreSQL – Application, analytical, integration or service workloads where PostgreSQL's open-source relational capabilities are appropriate.

MySQL – High-volume application workloads where the bank's chosen MySQL architecture and operational standards fit the use case.

Redis – Low-latency caching, session data, counters or other fast-access temporary/stateful workloads where an in-memory key-value store is appropriate.

Level 2 – Applying SQL

Q11.

SQL:

```
SELECT current_database() AS database_name,  
       current_user AS current_user,  
       current_setting('port') AS server_port;
```

Q12.

SQL:

```
SELECT ordinal_position,  
       column_name,  
       data_type,  
       is_nullable  
FROM information_schema.columns  
WHERE table_schema = 'public'  
      AND table_name = 'loan'  
ORDER BY ordinal_position;
```

```
SELECT COUNT(*) AS degree  
FROM information_schema.columns  
WHERE table_schema = 'public'  
      AND table_name = 'loan';
```

Q13.

SQL:

```
SELECT table_name,  
       (xpath('/row/c/text()',  
              query_to_xml(  
                  format('SELECT count(*) AS c FROM %I.%I', table_schema, table_name),  
                  false, true, ''  
              ))[1]::text::bigint AS row_count  
FROM information_schema.tables  
WHERE table_schema = 'public'  
      AND table_type = 'BASE TABLE'  
ORDER BY table_name;
```

Q14.

SQL:

```
SELECT  
  child_ns.nspname AS child_schema,  
  child.relname AS child_table,  
  child_col.attname AS child_column,
```

```

parent_ns.nspname AS parent_schema,
parent.relname AS parent_table,
parent_col.attname AS parent_column,
con.conname AS constraint_name
FROM pg_constraint con
JOIN pg_class child
  ON child.oid = con.conrelid
JOIN pg_namespace child_ns
  ON child_ns.oid = child.relnamespace
JOIN pg_class parent
  ON parent.oid = con.confrelid
JOIN pg_namespace parent_ns
  ON parent_ns.oid = parent.relnamespace
JOIN LATERAL unnest(con.conkey) WITH ORDINALITY AS ck(attnum, ord)
  ON TRUE
JOIN LATERAL unnest(con.confkey) WITH ORDINALITY AS pk(attnum, ord)
  ON pk.ord = ck.ord
JOIN pg_attribute child_col
  ON child_col.attrelid = child.oid AND child_col.attnum = ck.attnum
JOIN pg_attribute parent_col
  ON parent_col.attrelid = parent.oid AND parent_col.attnum = pk.attnum
WHERE con.contype = 'f'
ORDER BY child_table, child_column;

```

Q15.

SQL:

```

SELECT
  schemaname,
  relname AS table_name,
  pg_size_pretty(pg_total_relation_size(relid)) AS total_size,
  pg_total_relation_size(relid) AS total_size_bytes
FROM pg_catalog.pg_statio_user_tables
ORDER BY total_size_bytes DESC
LIMIT 3;

```

Q16. SQL:

BEGIN;

```

INSERT INTO customer
(customer_id, name, date_of_birth, email, phone, city,
kyc_status, risk_rating, customer_since, home_branch)
VALUES
(1032, 'Latha Krishnan', DATE '1985-09-23',
'latha.krishnan@example.com', '9845099887', 'Chennai',
'VERIFIED', 'MEDIUM', DATE '2026-03-01', 5);

```

```

INSERT INTO account
(account_id, account_number, customer_id, account_type,
balance, opened_date, interest_rate)
VALUES
(9047, 'ICIC0000009047', 1032, 'CURRENT',
500000.00, DATE '2026-03-01', 0.00);

```

```

SELECT * FROM customer WHERE customer_id = 1032;
SELECT * FROM account WHERE account_id = 9047;

```

COMMIT;

Q17. (Core)

SQL:

BEGIN;

```
UPDATE account
SET balance = balance + 75000
WHERE account_id = 9047;
```

```
UPDATE account
SET balance = balance - 20000
WHERE account_id = 9047;
```

```
SELECT account_id, account_number, balance
FROM account
WHERE account_id = 9047;
```

COMMIT;

Q18. (Core)

SQL:

-- First demonstrate the foreign-key protection:

```
DELETE FROM customer
WHERE customer_id = 1032;
```

-- PostgreSQL should reject this while the account row still references customer 1032.

-- Record the exact ERROR shown by your own database.

-- Correct order:

```
DELETE FROM account
WHERE account_id = 9047;
```

```
DELETE FROM customer
WHERE customer_id = 1032;
```

```
SELECT COUNT(*) AS customer_count FROM customer;
SELECT COUNT(*) AS account_count FROM account;
```

Q19.

SQL:

-- 1. Invalid account type

```
INSERT INTO account
(account_id, account_number, customer_id, account_type,
balance, opened_date, interest_rate)
VALUES
(9991, 'TEST0000009991', 1, 'FIXED',
1000.00, CURRENT_DATE, 0.00);
```

-- 2. Negative transaction amount

```
INSERT INTO account_txn
(/* use the actual required transaction columns */)
VALUES
(/* values matching the supplied schema, with amount = -5000 */);
```

```
-- 3. Invalid KYC status
INSERT INTO customer
(customer_id, name, date_of_birth, email, phone, city,
kyc_status, risk_rating, customer_since, home_branch)
VALUES
(9992, 'Constraint Test', DATE '1990-01-01',
'constraint.test@example.com', '9000000000', 'Test City',
'IN_PROGRESS', 'MEDIUM', CURRENT_DATE, 1);
```

Q20.

SQL:

```
SESSION A
BEGIN;
SELECT account_id, balance FROM account WHERE account_id = 9004;
UPDATE account
SET balance = balance + 1000
WHERE account_id = 9004;
SELECT account_id, balance FROM account WHERE account_id = 9004;
-- Leave this transaction open; do NOT COMMIT yet.
```

```
SESSION B
SELECT account_id, balance FROM account WHERE account_id = 9004;
-- This should still show the pre-update committed balance.
```

```
SESSION A
ROLLBACK;
```

```
SESSION B
SELECT account_id, balance FROM account WHERE account_id = 9004;
```

Q21.

SQL:

```
-- Record balances before
SELECT account_number, balance
FROM account
WHERE account_number IN ('9004', '9010')
ORDER BY account_number;
```

```
BEGIN;
```

```
UPDATE account
SET balance = balance - 12000
WHERE account_number = '9004';
```

```
UPDATE account
SET balance = balance + 12000
WHERE account_number = '9010';
```

```
-- Demonstrate the temporary transfer inside the transaction
SELECT account_number, balance
FROM account
WHERE account_number IN ('9004', '9010')
```

```
ORDER BY account_number;
```

```
ROLLBACK;
```

```
-- Prove neither balance changed  
SELECT account_number, balance  
FROM account  
WHERE account_number IN ('9004', '9010')  
ORDER BY account_number;
```

Q22.

SQL:

```
SHOW shared_buffers;  
SHOW work_mem;  
SHOW wal_level;  
SHOW fsync;
```

Level 3 – Analysis and Design Decisions

Q23.

I would reject MyISAM for a core UPI transaction workload because MyISAM does not provide transactional guarantees and uses table-level locking, creating unacceptable risks for concurrent financial updates and failure recovery. For MySQL, InnoDB is the appropriate transactional alternative because it supports transactions, row-level locking, crash recovery and foreign keys. Capacity for 4,000 transactions per second would still need to be validated with realistic load testing and an architecture designed for the required availability and scaling.

Q24.

One design has conflicting workload requirements: account transaction history needs fast point/range retrieval for a single account, while the risk team's monthly totals require large scans and aggregations across many accounts. I would keep the operational transaction store optimized for OLTP account-history queries and feed a separate analytical store/data warehouse for aggregated risk reporting. In this programme, the relational/SQL foundation is built in the database/SQL modules, while the later Redis and Kafka modules provide the technologies for low-latency access and event/data movement respectively; use the exact module mapping from the programme schedule when submitting.

Q25.

An application log saying “transfer successful” is only evidence that the application reached and logged that point; it does not prove that the database transaction committed or that both account balances were durably changed. The evidence should come from the database transaction/commit record and verified account rows, with matching debit and credit entries and the relevant transaction/audit identifiers. For an incident review, database-side evidence should be correlated with application logs rather than replaced by them.

Q26.

I would not allow fsync = off on the core banking PostgreSQL server because it weakens crash durability and can result in data loss or, after a severe failure, database corruption. The performance gain is not an acceptable trade-off for a system whose committed financial records must survive failures. If permitted at all, fsync = off should be restricted to disposable development/test environments or controlled rebuildable batch/staging systems where data can be regenerated and the server is not serving authoritative banking records.

Q27.

First, a database constraint protects the rule no matter which application, script, integration job or administrator attempts to write the data; an application-only check can be bypassed by another writer. Second, keeping fundamental integrity rules close to the data provides a final safety barrier and prevents invalid states from being committed concurrently. The application layer is genuinely the right place for rules that depend on user experience, workflow, external services or complex business context that cannot reasonably be expressed as a database constraint—for example, deciding which personalized offer to display to a customer.

Q28.

Four product differences that could create migration/merge issues are: (1) SQL dialect and built-in functions/syntax differ; (2) data types, identity/sequence behavior and date/time handling differ; (3) indexing, query-

planner behavior and optimizer features differ; and (4) transaction, locking, isolation, replication and backup/recovery mechanisms differ. In practice, these differences can affect DDL, stored procedures/functions, application SQL, performance, migration tooling and operational procedures, so they would need to be tested rather than assumed to be interchangeable.

Q29.

Moving from spreadsheets to a relational database will give the branch and other authorized teams a common source of customer and account information.

An update made to the customer record can then be used consistently by the systems that need that information.

The database can also enforce important rules, such as preventing duplicate identifiers and rejecting invalid values.

It can control simultaneous updates so two users do not accidentally overwrite each other's changes.

It can also treat a multi-step banking operation as one unit, so incomplete operations can be undone.

After a failure, the database has recovery mechanisms to restore committed information.

However, moving to a database will not automatically make incorrect business processes correct.

It will not fix poor source data, incorrect requirements, weak access controls, or badly written applications by itself.

The bank must therefore define data ownership and clear responsibilities for maintaining each field.

Teams must also use controlled application interfaces rather than maintaining private spreadsheet copies.

Access permissions, backups, monitoring, testing and audit procedures must be established and followed.

Users should be trained to work from the approved system of record.

Regular reconciliation should still be performed so that operational teams can detect unexpected differences.

The result is stronger control and consistency, but it requires changes in both technology and day-to-day working practices.